

Langage C

TD/TP n° 6: Les fichiers

Dr KENMOGNE EDITH BELISE

Questions de Cours

1) Discipliner l'accès aux fichiers

- 1.1) Est-il convenable que deux programmes utilisent simultanément un même fichier en écriture ?
- 1.2) Peut-on admettre que deux programmes utilisent simultanément le même fichier en lecture ?

2) Ouverture de fichier

Pour pouvoir manipuler un fichier, un programme a besoin d'un certain nombre d'informations : les adresses des zones du disque contenant le fichier, la position de la tête de lecture, le mode d'accès au fichier (lecture ou écriture) . . . Ces informations sont rassemblées dans une structure dont le type `FILE` défini dans `stdio.h`. **Un pointeur sur `FILE`, c'est à dire un objet de type `FILE*`, est appelé flot de données (en anglais, *stream*).**

`FILE* fopen(char *nom_physique_du_fichier, char *mode_ouverture)` : Elle prend en argument le nom physique du fichier et le mode d'ouverture et retourne un flot de données. Si l'exécution de cette fonction ne se déroule pas normalement, la valeur retournée est le pointeur `NULL`. Il est donc recommandé de toujours tester si la valeur renvoyée par la fonction `fopen` est égale à `NULL` afin de détecter les erreurs (lecture d'un fichier inexistant...).

2.1) Donner la chaîne de caractères qui exprime chaque mode d'ouverture de fichier qui suit : (1) le mode lecture (*read-mode*) , le mode écriture (*write-mode*) et le mode ajout (*append_mode*).

2.2) Donner la signification de chaque instruction suivante :

```
FILE *in_file, *out_file, *app_file ;
in_file = fopen("produit.txt","r");
out_file= fopen("etudiant.txt","w");
app_file= fopen("commande.txt","a");
```

3) Pourquoi les routines opérant sur les fichiers prennent en argument un `FILE*` et non un `FILE` ?

On rappelle que le type prédéfini `FILE` est une structure (*struct*) comprenant plusieurs champs. Un des champs contient *le nom physique du fichier*. D'autres champs décrivent la zone du disque contenant le fichier. Un autre champ indique *la position courante de lecture* dans le fichier si le fichier est ouvert en lecture, *la position courante d'écriture* dans le fichier si le fichier est ouvert en écriture ou *la position courante d'ajout* dans le fichier si le fichier est ouvert en mode *append*.

3.1) Citer au moins une information de `FILE` qui doit être modifiée pendant la lecture d'un fichier. Même question pour l'écriture, l'ajout et la fermeture.

3.3) Rappelons que le langage C utilise le passage par valeur. Que doit on passer en paramètre pour pouvoir modifier un objet `FILE` (un `FILE` ou un pointeur sur `FILE`, c'est à dire un `FILE*`) ?

3.4) Citer l'unique routine opérant sur les fichiers qui prend en argument *le nom physique de fichier*.

4) Lecture et ecriture des valeurs des types primitifs dans un fichier texte

`int fscanf(FILE *stream, char *format, ...)` : Si tout s'est bien passé, la valeur de retour est le nombre de valeurs lues. En cas d'erreur, la valeur de retour est `EOF` et la variable globale prédéfinie `errno` est positionnée correctement.

`int fprintf(FILE *stream, char *format, ...)` : Si tout s'est bien passé, la valeur de retour est le nombre de caractères que la fonction a écrit dans le fichier. En cas d'erreur, la valeur de retour est `EOF` et la variable globale prédéfinie `errno` est positionnée correctement.

4.1) Quel est l'intérêt de la fonction `fscanf` ? Donner la signification de `fscanf(in_file, "%f %d", &x, &m)`.

4.2) Mêmes questions qu'en 4.1) avec `fprintf` et `fprintf(out_file, "%s %s %s %d", "We", "are", "in", 2012)`.

5) Lecture et ecriture d'un caractère dans un fichier texte

`int fgetc(FILE *out_file)` : renvoie un entier, qui est soit un caractère, soit la valeur `EOF` si on se trouve à la fin du fichier. Dans le cas où on a atteint la fin du fichier, la variable globale prédéfinie `errno` est

positionnée correctement.

int fputc(int ch, FILE *out_file) : Le int *ch* est converti en un caractère et écrit dans le fichier. La valeur retournée est *ch* si tout s'est bien passé. Cependant, en cas d'erreur d'exécution, la valeur retournée est *EOF* et la variable globale prédéfinie *errno* est positionnée correctement.

5.1) Dire ce que fait l'instruction suivante : *ch=fgetc(in_file)*

5.2) Même question qu'en 5.1) avec *fputc('a', out_file)* »

6) Lecture et écriture d'une chaîne de caractères dans un fichier texte

char* fgets(char *chaîne, int taille, FILE *in_file) : La fonction lit au plus (*taille - 1*) caractères et les range dans le tableau pointé par *chaîne*. *fgets* lit les caractères du fichier et les range dans le tableau pointé par *chaîne* jusqu'à la rencontre d'un *line-feed* (qui est mis dans le tableau), ou la rencontre de *fin de fichier*, ou jusqu'à ce qu'il ne reste plus qu'un seul caractère libre dans le tableau. *fgets* complète alors les caractères lus par un caractère null. ***fgets peut donc servir indifféremment à lire une ligne ou une chaîne quelconque.*** La fonction *fgets* retourne le pointeur chaîne cas de lecture sans erreur, ou NULL dans le cas de fin de fichier ou d'erreur.

int fputs(char *chaîne, FILE *out_file) : *fputs* écrit sur le fichier le contenu du tableau dont la fin est indiquée par un caractère null '\0'. Le tableau de caractères peut contenir ou non un *line-feed*. ***fputs peut donc servir indifféremment à écrire une ligne ou une chaîne quelconque.*** La fonction *fputs* retourne une valeur non négative si l'écriture se passe sans erreur, et EOF en cas d'erreur.

6.1) Même question qu'en 5.1) avec *fgets(ligne, 200, in_file)*

6.1) Même question qu'en 5.1) avec *fputs(ligne, 200, out_file)*

7) Fin de fichier et fermeture de fichier

8.1) Quelle est la différence entre EOF et la routine *int feof(FILE *in_file)* ?

8.3) Mêmes questions qu'en 4.1) avec *fclose* et *fclose(out_file)*.

Exercice 1 : Analyse de programme

<pre>#include <stdio.h> int nbre_caracteres(char *nom_physique) { FILE *in_file; int cpt=0 ; in_file = fopen(nom_physique,"r"); while (fgetc(in_file)!=EOF) cpt++; fclose(in_file) ; return cpt ; } void main () { char np[25] ; int nbre ; printf("Entrer le nom de fichier texte :) " ; scanf("%s ", np) ; nbre=nbre_caracteres(np) ; printf("Resultat :%d", nbre) ; return ; }</pre>	<pre>#include <stdio.h> void nbre_lignes(char *nom_physique, int *cpt_min, int *cpt_maj) { FILE *in_file; int ch ; in_file = fopen(nom_physique,"r"); *cpt_min=0 ; *cpt_maj=0 ; while (! feof(in_file)) { ch=fgetc(in_file) ; if (('a' <= ch) && (ch <= 'z')) *cpt_min++ ; else if (('A' <= ch) && (ch <= 'Z')) *cpt_maj++ ; } fclose(in_file) ; return ; }</pre>
Programme a	Programme b

1) Qu'est ce que fait le programme a ? Proposer une version équivalente du test de la boucle while qui utilise la routine eof et non EOF.

2) Qu'est ce que fait le programme b ? Proposer une version équivalente du test de la boucle while qui utilise EOF et non la routine feof.

3) Pourquoi la fonction nbre_lignes ne peut pas s'exécuter correctement si on remplace toutes les occurrences de *cpt_min par cpt_min et toutes les occurrences de *cpt_maj par cpt_maj.

4) Ecrire une fonction principale main qui utilise la fonction du programme b.

Exercice 2 : Nombre de lignes d'un fichier texte

Ecrire une fonction *int nbre_lignes(char *np_source)* qui retourne le nombre de lignes du fichier de nom **np_source*.

Exercice 3 : Copie de fichier texte

Ecrire une fonction *int copie(char *np_cible, char *np_source)* qui crée le fichier de nom **np_cible* et copie le contenu du fichier **np_source* dans le fichier créée.

Probleme 1

Nous considérons ici un fichier texte dont chaque ligne comprend : (1) le matricule d'un étudiant, (2) le nom de l'étudiant, et (3) quatre notes. Cette structure est illustrée comme suit :

m0000 Messi 15 17 12 16

m0001 Tamo 17 18 11 15

m0002 Douala 16 15 15 14

m0003 Tang 9 14 5 17

1) Ecrire une fonction *int nombre(char *nom_fichier)* qui prend en argument le nom physique d'un fichier respectant la structure ci-dessus et retourne nombre d'étudiants.

2) Ecrire une fonction *int affiche(char *nom_fichier)* qui lit le fichier et affiche toutes les lignes à l'écran.

3) Ecrire une fonction *int aff_moy(char *nom_fichier)* qui lit le fichier et affiche à l'écran, le matricule, le nom et la moyenne de chaque étudiant.

4) Ecrire une fonction *float moy_gen(char *nom_fichier)* qui lit le fichier et retourne la moyenne générale.

5) Définir une structure *etudiant* (*struct etudiant*) comprenant : (1) un champ pour le matricule, (2) un champ pour le nom, et (3) un champ pour les quatre notes.

6) Ecrire une fonction *struct etudiant* lire()* qui lit au clavier des informations sur un étudiant, les stocke dans une structure créée dynamiquement et retourne un pointeur sur la dite structure.

7) Ecrire une fonction *int ajout(struct etudiant e, char *nom_fichier)* qui ajoute un étudiant en fin de fichier.

8) Ecrire une fonction *struct etudiant* to_vector(char *nom_fichier, int *size)* qui stocke tous les étudiants du fichier dans un vecteur créé dynamiquement, écrit la taille du vecteur dans **taille* et retourne ledit vecteur.

9) Ecrire une fonction *int to_file(char *nom_fichier, int size, struct etudiant *vect)* qui crée un fichier et stocke tous les étudiants du vecteur *vect* de taille **size* dans le fichier créée.

10) Ecrire une fonction *void trie(struct etudiant *vect, int size)* qui trie le vecteur *vect* par ordre croissant sur les moyennes.

11) Ecrire une fonction *void aff_vect(struct etudiant *vect, int size)* qui affiche les étudiants à l'écran (ligne par ligne).

12) Ecrire une fonction *int aff_croissant(char *nom_fichier)* qui lit le fichier et affiche toutes les lignes à l'écran par ordre de moyenne croissant.

13) Tester toutes ces fonctions.